CS202 Design Pattern Report: Interpreter Pattern

Formal Grammars, Abstract Syntax Trees, and Modern C++ Memory Architectures

Group 52
Nguyễn Đình Minh Huy (Student ID: 25125083)
Trần Gia Huy (Student ID: 25125084)

August 21, 2026

Contents

1 Real-world Problem & Theoretical Motivation

1.1 Background and Domain-Specific Languages (DSLs)

In software engineering, systems frequently encounter requirements to interpret, evaluate, or execute statements written in domain-specific mini-languages at runtime. Unlike general-purpose programming languages (e.g., C++, Java, Rust), domain-specific languages (DSLs) are tailored to specialized problem domains. Prominent examples include:

- **Dynamic Business Rule Engines:** Validating loan eligibility, risk scoring, or discount tiers defined by non-developer business analysts.
- **Database Query and Filter Parsers:** Evaluating SQL-like `WHERE` clauses (e.g., `age > 21 AND status == 'ACTIVE'`) dynamically generated from user search interfaces.
- **IoT Sensor Event Processing:** Filtering high-throughput telemetry streams against custom user-defined logic conditions (e.g., `temp > 85.0 OR (pressure > 120.0 AND NOT valve_open)`).
- **Configuration-Driven Access Control (RBAC/ABAC):** Evaluating permission predicates such as `(isOwner OR isAdmin) AND NOT isBanned`.

Hardcoding every permutation of logical rules into static source code results in severe inflexibility. Every rule change requires recompilation, regression testing, and deployment. Consequently, systems require a mechanism to define, compose, and evaluate arbitrary sentences of a well-defined language dynamically at runtime.

1.2 Problem Statement: Boolean Logic Evaluator

To illustrate the problem concretely, consider a configurable logic engine that evaluates Boolean expressions consisting of:

1. **Variables:** Identifiers representing dynamic state (e.g., A, B, C).
2. **Constants:** Literal truth values (`true`, `false`).
3. **Unary Operators:** Logical negation (`NOT` / $\neg$).
4. **Binary Operators:** Conjunction (`AND` / $\wedge$), Disjunction (`OR` / $\vee$), and Exclusive-OR (`XOR` / $\oplus$).
5. **Groupings:** Nested parentheses controlling operator precedence.

The engine must take an expression along with a symbol table (a *Context* mapping identifiers to their Boolean values) and compute the exact truth result according to standard mathematical logic.

1.3 Language Grammar and Operator Rules

To keep the design clear and practical, the language is defined by simple composition rules rather than formal compiler notation:

- **Atomic Expressions (Terminals):** Either a named variable (e.g., A, B) looked up from the `Context` or a literal Boolean constant (`true`, `false`).
- **Unary Operation:** Negation (`NOT`) operating on a single child expression.
- **Binary Operations:** Conjunction (`AND`), Disjunction (`OR`), and Exclusive-OR (`XOR`) combining two child sub-expressions.
- **Operator Precedence:** Evaluated in standard algebraic order: NOT $>$ AND $>$ XOR $>$ OR, with nested sub-expressions enclosed in parentheses evaluated first.

By organizing expressions according to these simple rules, any valid logical statement can be translated directly into a hierarchical tree structure.

2 Naive Solution (Without Design Pattern)

In a naive C++ implementation, expressions are treated as raw text strings. The evaluation is performed through manual string searching, linear token splitting, or ad-hoc procedural loops.

```cpp
// Context: Maps variable names to boolean values
class Context {
public:
    void assign(const std::string& name, bool value) { variables_[name] = value; }
    bool lookup(const std::string& name) const {
        auto it = variables_.find(name);
        if (it == variables_.end()) throw std::runtime_error("Unknown variable: " + name);
        return it->second;
    }
private:
    std::map<std::string, bool> variables_;
};

class NaiveLogicEvaluator {
public:
    // Evaluates flat left-to-right expressions (e.g. "A AND B OR C")
    static bool evaluate(const std::string& expression, const Context& context) {
        std::vector<std::string> tokens = tokenize(expression);
        if (tokens.empty()) return false;

        bool result = parseOperand(tokens[0], context);

        for (size_t i = 1; i + 1 < tokens.size(); i += 2) {
            const std::string& op = tokens[i];
            bool rhs = parseOperand(tokens[i + 1], context);

            if (op == "AND") {
                result = result && rhs;
            } else if (op == "OR") {
                result = result || rhs;
            } else if (op == "XOR") {
                result = result ^ rhs;
            } else {
                throw std::invalid_argument("Unsupported operator: " + op);
            }
        }
        return result;
    }

private:
    static bool parseOperand(const std::string& token, const Context& context) {
        if (token == "true" || token == "1") return true;
        if (token == "false" || token == "0") return false;
        return context.lookup(token);
    }

```

```
47      static std::vector<std::string> tokenize(const std::string&
        expression) {
48          std::stringstream ss(expression);
49          std::string token;
50          std::vector<std::string> tokens;
51          while (ss >> token) tokens.push_back(token);
52          return tokens;
53      }
54  };
```

Listing 1: Naive Boolean Expression Evaluator (`naive.cpp`)

2.1 Step-by-Step Execution of the Naive Evaluator

To illustrate how the naive solution operates, consider evaluating the string `"A OR B AND C"` with context $A = \text{true}, B = \text{false}, C = \text{true}$:

1. **Tokenization (String Splitting):** The evaluator parses the raw string by whitespace into a linear token vector:

$$\texttt{tokens} = [\text{"A"}, \text{"OR"}, \text{"B"}, \text{"AND"}, \text{"C"}]$$

2. **Initial Left Operand Lookup:** The first token `"A"` is looked up in `Context` ($A = \text{true}$) to initialize the running accumulator `result = true`.

3. **Linear Left-to-Right Evaluation Loop:** The loop steps through consecutive operator-operand pairs:

   - *Step 3a:* Reads operator `"OR"` and operand `"B"` ($B = \text{false}$). Updates:

$$\texttt{result} = \texttt{result} \vee B = \text{true} \vee \text{false} = \textbf{true}$$

   - *Step 3b:* Reads operator `"AND"` and operand `"C"` ($C = \text{true}$). Updates:

$$\texttt{result} = \texttt{result} \wedge C = \text{true} \wedge \text{true} = \textbf{true}$$

Critical Flaw Demonstrated: The naive evaluator computes $(A \vee B) \wedge C = (\text{true} \vee \text{false}) \wedge \text{true} = \text{true}$. If the input were `"false OR true AND false"`, the mathematically correct precedence $A \vee (B \wedge C)$ yields $\text{false} \vee (\text{true} \wedge \text{false}) = \textbf{false}$, but the naive left-to-right evaluator incorrectly outputs $(\text{false} \vee \text{true}) \wedge \text{false} = \textbf{false}$ (or in other expressions, completely corrupts truth values due to lack of precedence and parenthesis support).

3 In-Depth Analysis of Naive Solution Drawbacks

The naive string-scanning and procedural dispatch approach presents several architectural, correctness, and performance limitations:

Deficiency Area	Manifestation in Naive Code	Architectural Consequence
Open-Closed Principle (OCP) Violation	Adding an operator (e.g., `NAND`, `IMPLIES`) requires modifying the central `if-else` loop in `evaluate()`.	Existing evaluation code must be modified, increasing regression risk.
Single Responsibility Principle (SRP) Violation	`NaiveLogicEvaluator` handles lexical tokenization, semantic validation, operator precedence, and evaluation in one class.	Parsing and evaluation logic cannot be tested or maintained independently.
Operator Precedence Limitations	Evaluates flat expressions strictly left-to-right. `A OR B AND C` evaluates as $(A \lor B) \land C$ instead of $A \lor (B \land C)$.	Produces incorrect results for expressions requiring operator precedence.
Nesting and Parentheses Support	Supporting nested subexpressions (e.g., `(A AND (NOT B)) OR C`) requires recursive regex or complex stack tracking.	Significantly increases procedural string manipulation complexity.
Evaluation Overhead	String allocations, repeated `substr()` calls, and linear scans incur additional overhead on repeated evaluations.	Inefficient when expressions are evaluated repeatedly against changing contexts.

Table 1: Design and Performance Limitations of the Naive Solution

4 Introduction to the Interpreter Design Pattern

4.1 Definition and Intent

According to the Gang of Four (GoF), the **Interpreter Pattern** is defined as:

> *"Given a language, define a representation for its grammar along with an interpreter that uses the representation to interpret sentences in the language."*

The pattern addresses the problem by representing each grammar rule as a class. The syntax of an expression is represented as an **Abstract Syntax Tree (AST)**, where each node is an instance of an expression class. Evaluation is performed by traversing the tree recursively.

4.2 Key Structural Components

- **Context (`Context`):** Encapsulates the global execution environment, maintaining state such as variable bindings (symbol table) needed by terminal expressions.

- **AbstractExpression** (`BooleanExpression`)**:** Declares an abstract interface with an evaluation method:

```
virtual bool interpret(const Context& context) const = 0;
```

- **TerminalExpression** (`VariableExpression`, `ConstantExpression`)**:** Represents leaf nodes corresponding to terminal grammar symbols. Directly evaluates based on the provided `Context`.
- **NonterminalExpression** (`AndExpression`, `OrExpression`, `NotExpression`, `XorExpression`)**:** Represents internal nodes of the AST corresponding to grammar production rules. Contains child `AbstractExpression` instances and aggregates their recursive evaluations.
- **Client:** Assembles (or receives from a parser) the composite AST hierarchy and initiates evaluation by invoking `interpret()` on the root node.

5 UML Class Diagrams

5.1 General GoF Interpreter Pattern Structure

The Interpreter pattern defines a polymorphic hierarchy where non-terminal composite nodes hold references to abstract expressions, while terminal nodes evaluate directly against the context.

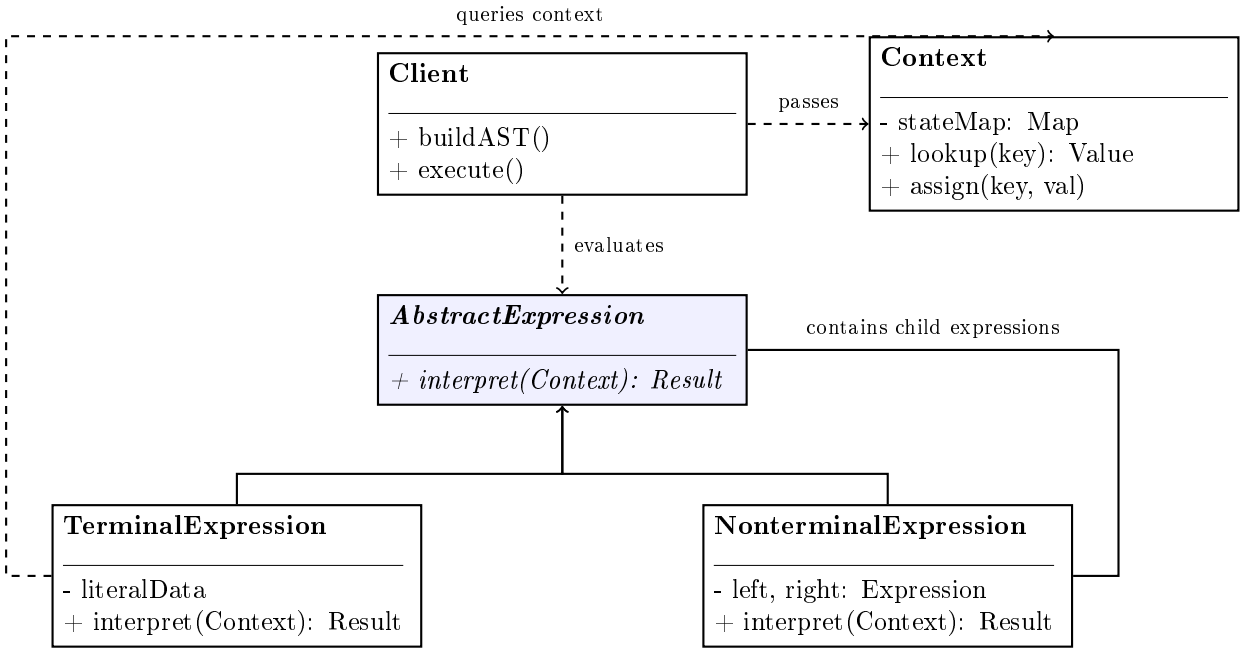

Figure 1: General GoF Interpreter Pattern UML Structure

5.2 Boolean Logic Evaluator Class Hierarchy

For our Boolean expression engine, `BooleanExpression` acts as the abstract component interface with concrete terminals (`VariableExpression`, `ConstantExpression`) and non-terminals owning child expressions via `std::unique_ptr`.

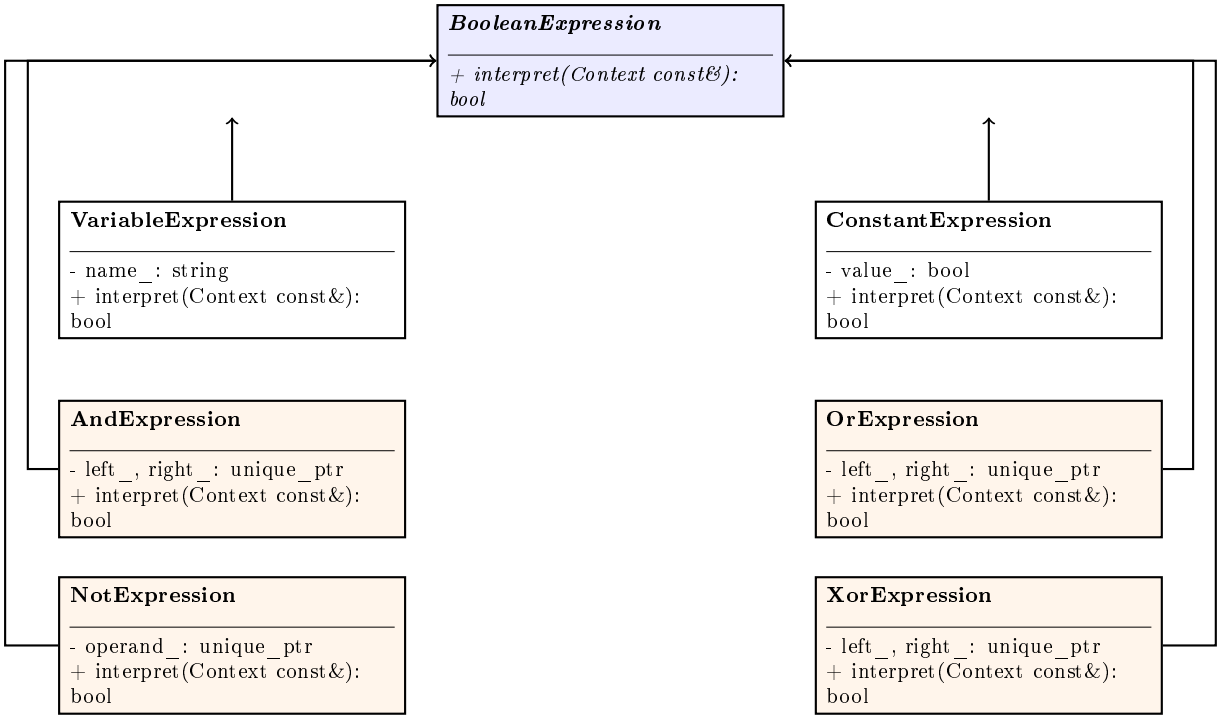

Figure 2: Concrete Boolean Logic Evaluator Class Hierarchy

6 Pattern-Based Implementation in C++17

6.1 Architectural Design Rationale: Strict AST vs. ASG/DAG

A key design decision in our C++ implementation is enforcing a strict **Abstract Syntax Tree (AST)** rather than an **Abstract Semantic Graph (ASG / DAG)**:

- **Strict Tree Structure (`std::unique_ptr`):** Every expression node has exactly *one unique parent*. Duplicate occurrences of a variable (such as A appearing in multiple sub-clauses) are instantiated as distinct terminal leaves. This is enforced using `std::unique_ptr` with move semantics, eliminating reference counting overhead and ensuring deterministic single-owner memory reclamation.
- **Graph Alternative (`std::shared_ptr`):** In optimizing engines with Common Subexpression Elimination (CSE), identical sub-expressions share node instances with multiple parents (a DAG). However, this introduces atomic reference counting overhead and cache complexity.

6.2 Source Code Implementation

The following listings demonstrate the complete, robust implementation using C++17 move semantics and unique pointers (`Interpreter/src/pattern.cpp`).

```
#include <iostream>
#include <string>
#include <map>
#include <memory>
#include <stdexcept>

```

```cpp
namespace pattern {

// Context: Holds runtime symbol bindings
class Context {
public:
    void assign(const std::string& name, bool value) {
        variables_[name] = value;
    }

    bool lookup(const std::string& name) const {
        auto it = variables_.find(name);
        if (it == variables_.end()) {
            throw std::runtime_error("Unbound variable in Context: " + name);
        }
        return it->second;
    }

private:
    std::map<std::string, bool> variables_;
};

// Abstract Expression Interface
class BooleanExpression {
public:
    virtual ~BooleanExpression() = default;
    virtual bool interpret(const Context& context) const = 0;
};
```

Listing 2: Context and Abstract Expression Interface

```cpp
// Terminal Expression: Variable Lookup
class VariableExpression : public BooleanExpression {
public:
    explicit VariableExpression(std::string name) : name_(std::move(name)) {}

    bool interpret(const Context& context) const override {
        return context.lookup(name_);
    }

private:
    std::string name_;
};

// Terminal Expression: Boolean Literal Constant
class ConstantExpression : public BooleanExpression {
public:
    explicit ConstantExpression(bool value) : value_(value) {}

    bool interpret(const Context&) const override {
        return value_;
    }

private:
    bool value_;
};
```

Listing 3: Terminal Expressions: Variables and Constants

```
// Non-Terminal: Logical Conjunction (AND)
class AndExpression : public BooleanExpression {
public:
    AndExpression(std::unique_ptr<BooleanExpression> left, std::unique_ptr<BooleanExpression> right)
        : left_(std::move(left)), right_(std::move(right)) {}

    bool interpret(const Context& context) const override {
        return left_->interpret(context) && right_->interpret(context);
    }

private:
    std::unique_ptr<BooleanExpression> left_;
    std::unique_ptr<BooleanExpression> right_;
};

// Non-Terminal: Logical Disjunction (OR)
class OrExpression : public BooleanExpression {
public:
    OrExpression(std::unique_ptr<BooleanExpression> left, std::unique_ptr<BooleanExpression> right)
        : left_(std::move(left)), right_(std::move(right)) {}

    bool interpret(const Context& context) const override {
        return left_->interpret(context) || right_->interpret(context);
    }

private:
    std::unique_ptr<BooleanExpression> left_;
    std::unique_ptr<BooleanExpression> right_;
};

// Non-Terminal: Logical Negation (NOT)
class NotExpression : public BooleanExpression {
public:
    explicit NotExpression(std::unique_ptr<BooleanExpression> operand)
        : operand_(std::move(operand)) {}

    bool interpret(const Context& context) const override {
        return !operand_->interpret(context);
    }

private:
    std::unique_ptr<BooleanExpression> operand_;
};

// Non-Terminal: Exclusive-OR (XOR)
class XorExpression : public BooleanExpression {
public:
    XorExpression(std::unique_ptr<BooleanExpression> left, std::unique_ptr<BooleanExpression> right)
        : left_(std::move(left)), right_(std::move(right)) {}

    bool interpret(const Context& context) const override {
        return left_->interpret(context) ^ right_->interpret(context);
    }

private:
```

```
56      std::unique_ptr<BooleanExpression> left_;
57      std::unique_ptr<BooleanExpression> right_;
58  };
59
60  } // namespace pattern
```

Listing 4: Non-Terminal Expressions: AND, OR, NOT, XOR with Unique Ownership

6.3 Execution Walkthrough & Step-by-Step AST Evaluation Trace

Consider the composite logical expression:

$$\text{Expression} = (A \wedge (\neg B)) \vee (C \oplus A) \tag{1}$$

Given the runtime variable bindings in Context:

$$A = \textbf{true}, \quad B = \textbf{false}, \quad C = \textbf{true}$$

```
void runPatternDemo() {
    pattern::Context context;
    context.assign("A", true);
    context.assign("B", false);
    context.assign("C", true);

    // Build AST: (A AND (NOT B)) OR (C XOR A)
    auto notB = std::make_unique<pattern::NotExpression>(
        std::make_unique<pattern::VariableExpression>("B")
    );
    auto aAndNotB = std::make_unique<pattern::AndExpression>(
        std::make_unique<pattern::VariableExpression>("A"),
        std::move(notB)
    );
    auto cXorA = std::make_unique<pattern::XorExpression>(
        std::make_unique<pattern::VariableExpression>("C"),
        std::make_unique<pattern::VariableExpression>("A")
    );
    auto fullExpr = std::make_unique<pattern::OrExpression>(
        std::move(aAndNotB),
        std::move(cXorA)
    );

    bool result = fullExpr->interpret(context);
    std::cout << "Result: " << (result ? "true" : "false") << "\n"; //
   Outputs: true
}
```

Listing 5: Building and Evaluating the AST

Table 2 summarizes the post-order depth-first traversal and evaluation sequence of the AST nodes:

Step	Node Evaluated	Subtree	Value	Explanation / Operation
1	`VariableExpression("B")`	B	**false**	Context lookup returns $B = \text{false}$.
2	`NotExpression`	$\neg B$	**true**	Negation: !false = true.
3	`VariableExpression("A")`	A	**true**	Context lookup returns $A = \text{true}$.
4	`AndExpression`	$A \wedge (\neg B)$	**true**	Conjunction: true $\wedge$ true = true.
5	`VariableExpression("C")`	C	**true**	Context lookup returns $C = \text{true}$.
6	`VariableExpression("A")`	A	**true**	Context lookup returns $A = \text{true}$.
7	`XorExpression`	$C \oplus A$	**false**	Exclusive OR: true$\oplus$true = false.
8	`OrExpression` (Root)	$(A \wedge \neg B) \vee (C \oplus A)$	**true**	Root Disjunction: true $\vee$ false = **true**.

Table 2: Recursive Post-Order Traversal Trace for AST Evaluation

7 Evaluation of Trade-offs (Pros and Cons)

7.1 Advantages

- **High Grammar Extensibility (OCP Compliance):** Introducing new language constructs or operators (e.g., adding an `ImpliesExpression` for $P \implies Q \equiv \neg P \vee Q$) requires creating a new subclass of `BooleanExpression` without altering any existing classes.
- **Single Responsibility Principle (SRP):** Each class is strictly responsible for interpreting exactly one grammar symbol.
- **Decoupled Syntax Structure from Execution:** The structural representation (AST) is cleanly separated from the runtime evaluation state (`Context`).
- **Zero Evaluation Overhead with `std::unique_ptr`:** By relying on direct polymorphic virtual dispatch and unique pointers, tree evaluation avoids string-slicing allocations and runs at native CPU speed.

7.2 Disadvantages and Limitations

- **Combinatorial Class Explosion for Large Grammars:** For complex programming languages with hundreds of grammar production rules (e.g., full ECMAScript or SQL-92), maintaining hundreds of distinct AST classes manually becomes unmanageable.
- **Call-Stack Depth and Recursion Overhead:** Deeply nested syntax trees can lead to stack overflow if evaluated via naive recursion on resource-constrained embedded systems.

- **Grammar Maintenance Overhead:** Modifying operator precedence rules requires structural modifications to the AST assembly rather than simple configuration changes.

8 Modern Web and Mobile Engineering Applications

8.1 Web Development Applications

- **Dynamic URL Routing Engines:** Modern web frameworks (e.g., Express.js, Fastify, React Router) translate parameterized route patterns (e.g., `/api/v1/users/:id/orders/`) into regex/AST matchers that extract path parameters from incoming HTTP request strings.
- **CSS Selector Engines:** Browser engines (e.g., WebKit, Blink) parse complex compound CSS selectors (e.g., `div.container > article:first-child p.lead`) into selector ASTs to efficiently match DOM nodes during style resolution.
- **Client-Side Formula Parsers:** Web spreadsheet applications (e.g., Google Sheets, Luckysheet) parse formula expressions (e.g., `=SUM(A1:A10) * IF(B1 > 50, 1.1, 1.0)`) into syntax trees that re-evaluate reactively upon cell updates.

8.2 Mobile Development Applications

- **ICU MessageFormat & Pluralization Parsers:** Mobile operating systems (iOS and Android) interpret dynamic localization strings containing gender, cardinal numbers, and pluralization branches (e.g., `"{count, plural, =0{No items} =1{One item} other{# items}}"`).
- **Server-Driven UI (SDUI) Conditional Rendering:** Applications driven by backend layout schemas interpret JSON condition trees (e.g., `{"and": [{"user.isVip": true}, {"device.osVersion": "> 14"}]}`) to dynamically show or hide mobile screen widgets.

9 Advanced Architectural Combinations: Observer and Mediator Integration

In real-world software architectures, design patterns rarely exist in isolation. The Interpreter pattern naturally combines with the Observer and Mediator patterns to create reactive, event-driven, and highly decoupled systems.

9.1 Combining Interpreter with the Observer Pattern: Reactive Logic Evaluation

In a static Interpreter implementation, clients must manually invoke `interpret()` whenever variable values change. In modern reactive architectures—such as reactive UI frameworks, spreadsheet dependency graphs, or IoT sensor triggers—this workflow is refactored by applying the Observer pattern.

The evaluation `Context` functions as an observable Subject. When a variable's value is modified (for example, via `Context::assign("temperature", 92.5)`), the Context publishes a state-change notification. Root expressions and dependent components register as Observers listening for specific variable keys. Upon receiving an update event, they automatically trigger re-interpretation of their AST and emit new evaluation results reactively, eliminating wasteful CPU polling loops.

9.2 Combining Interpreter with the Mediator Pattern: Decoupled Command Coordination

In complex distributed systems, such as air traffic control systems, command dispatchers, or workflow engines, components must coordinate without direct peer-to-peer coupling. Here, the Mediator pattern leverages an embedded Interpreter engine.

Colleague components do not interact directly with one another. Instead, they send declarative query or command expressions to a central `Mediator`. The central Mediator uses an internal Interpreter engine to dynamically parse incoming expressions, evaluate business rules, and check permission predicates before routing actions to target colleagues. This decouples the communication topology into an $O(N)$ star architecture while retaining the flexibility to modify business coordination rules dynamically at runtime.

10 Conclusion

The Interpreter Design Pattern provides an elegant, object-oriented solution for representing and evaluating dynamic domain-specific grammars. By modeling production rules as distinct classes and organizing them into an Abstract Syntax Tree (AST), the pattern achieves strict adherence to the Open-Closed and Single Responsibility Principles.

Furthermore, combining the Interpreter pattern with modern C++17 memory management idioms—specifically using `std::unique_ptr` for exclusive node ownership and zero-overhead move semantics—yields a high-performance, memory-safe execution engine. When combined with the Observer and Mediator patterns, it forms the backbone for reactive rule engines and decoupled, dynamic event-driven architectures.

References

- Gamma, E., Helm, R., Johnson, R., & Vlissides, J. (1994). *Design Patterns: Elements of Reusable Object-Oriented Software*. Addison-Wesley.
- Aho, A. V., Lam, M. S., Sethi, R., & Ullman, J. D. (2006). *Compilers: Principles, Techniques, and Tools* (2nd ed.). Addison-Wesley.
- Meyers, S. (2014). *Effective Modern C++: 42 Specific Ways to Improve Your Use of C++11 and C++14*. O'Reilly Media.
- Refactoring.Guru. *Design Patterns: Interpreter Pattern*. `https://refactoring.guru/design-patterns/interpreter`